Transcript.IO

Transcript.IO est un site de prise de notes. On peut écrire une note au clavier ou la dicter au micro. La dictée est transcrite par Whisper directement dans le navigateur. Les notes sont enregistrées dans une base de données.

1. Technologies utilisées

J'ai fait le site avec **Next.js 16** (React et TypeScript). Next.js permet d'avoir la partie client et la partie serveur dans le même projet : la page lit la base côté serveur, et les modifications passent par des Server Actions.

Le style est fait avec **Tailwind CSS**, avec quelques composants de base de shadcn/ui (bouton, champ, liste déroulante).

La base de données est **SQLite**, utilisée avec Drizzle ORM. La base est un simple fichier créé au premier lancement à partir de `db/init.sql`, il n'y a donc pas de serveur de base de données à installer. Elle contient 2 tables :

- `tags` : id, name_fr, name_en (les étiquettes, avec leur nom dans les deux langues) ;
- `notes` : id, title, content, dictated, tag_id, created_at, updated_at.

Le site fait un `SELECT` avec une jointure pour afficher la liste des notes, et des `INSERT`, `UPDATE` et `DELETE` pour ajouter, modifier et supprimer une note.

Nombre de fichiers. La partie client contient 15 fichiers : `app/layout.tsx`, `app/page.tsx`, `app/globals.css`, 5 composants dans `components/`, 5 composants dans `components/ui/`, `hooks/use-whisper.ts` et `lib/i18n.ts`. La partie serveur contient 10 fichiers : `app/actions.ts`, `lib/prefs.ts`, les 3 fichiers de `db/`, et 5 fichiers de configuration obligatoires avec Next.js et npm (`package.json`, `tsconfig.json`, `next.config.ts`, `postcss.config.mjs`, `.npmrc`). Le fichier `package-lock.json` est généré automatiquement par npm.

2. Utilisation du site

La page principale

Le site tient sur une seule page. En haut, il y a le titre et trois boutons : **EN** pour passer le site en anglais, **Version mobile** pour passer à la version téléphone, et un bouton pour le thème sombre. En dessous, il y a le formulaire pour écrire une note, puis la liste des notes, de la plus récente à la plus ancienne.

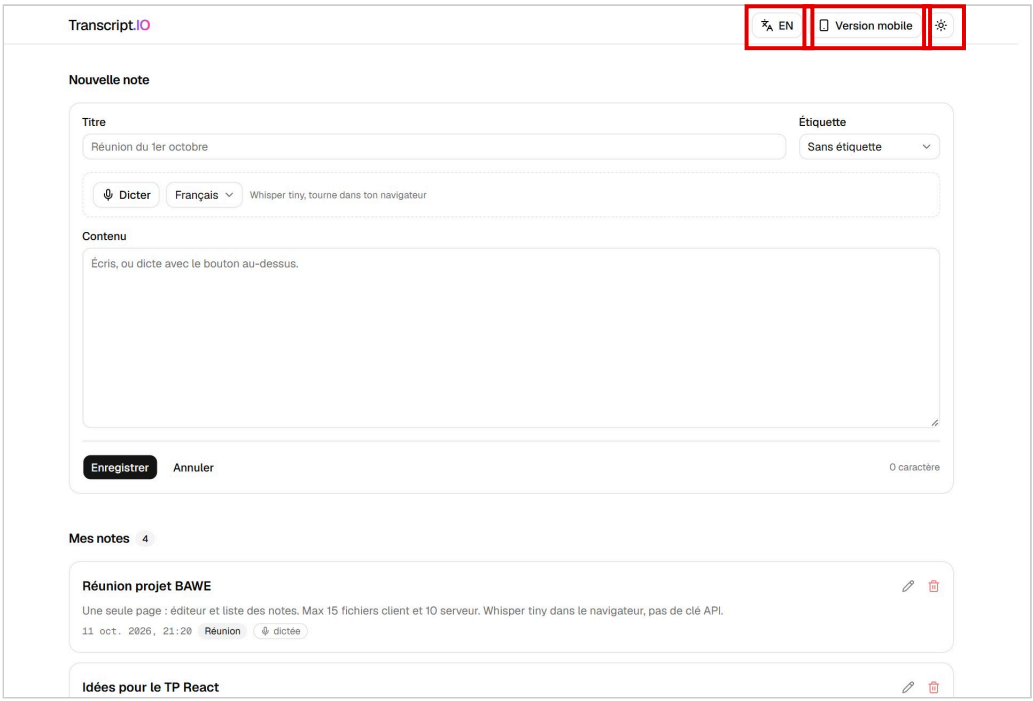


Figure 1 : version bureau en français, avec les boutons de l'en-tête

Ajouter une note

On écrit un titre (obligatoire), on choisit une étiquette si on veut (Cours, Idées, Réunion ou Perso), puis on écrit le contenu. On clique ensuite sur **Enregistrer** : la note est ajoutée dans la base et apparaît en haut de la liste.

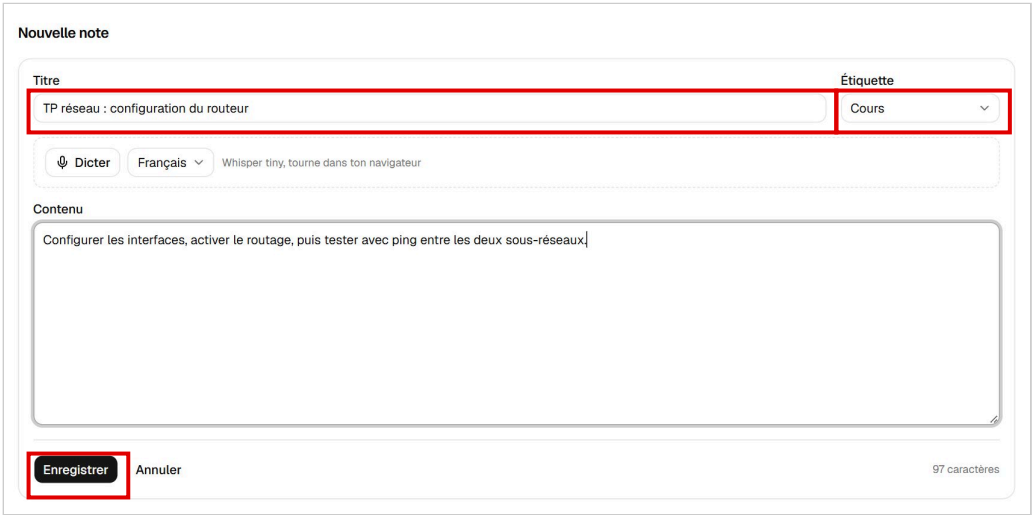


Figure 2 : formulaire rempli avant l'enregistrement

Dicter une note

Le bouton **Dicter** lance la dictée. La première fois, le modèle Whisper est téléchargé (une seule fois, il est gardé en cache). Le navigateur demande ensuite l'autorisation d'utiliser le micro. On parle, puis on clique sur **Arrêter**. Le texte est ajouté à la fin du contenu et la note est marquée « dictée ».



Figure 3 : téléchargement du modèle

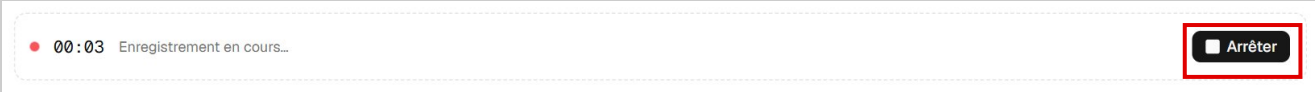


Figure 4 : enregistrement en cours



Figure 5 : transcription

Modifier ou supprimer une note

Le crayon sur une note la charge dans le formulaire, qui passe en « Modifier la note ». Il suffit de changer ce qu'on veut puis d'enregistrer. La corbeille devient un bouton **Confirmer** : il faut cliquer une deuxième fois pour supprimer, ce qui évite de supprimer une note par erreur.

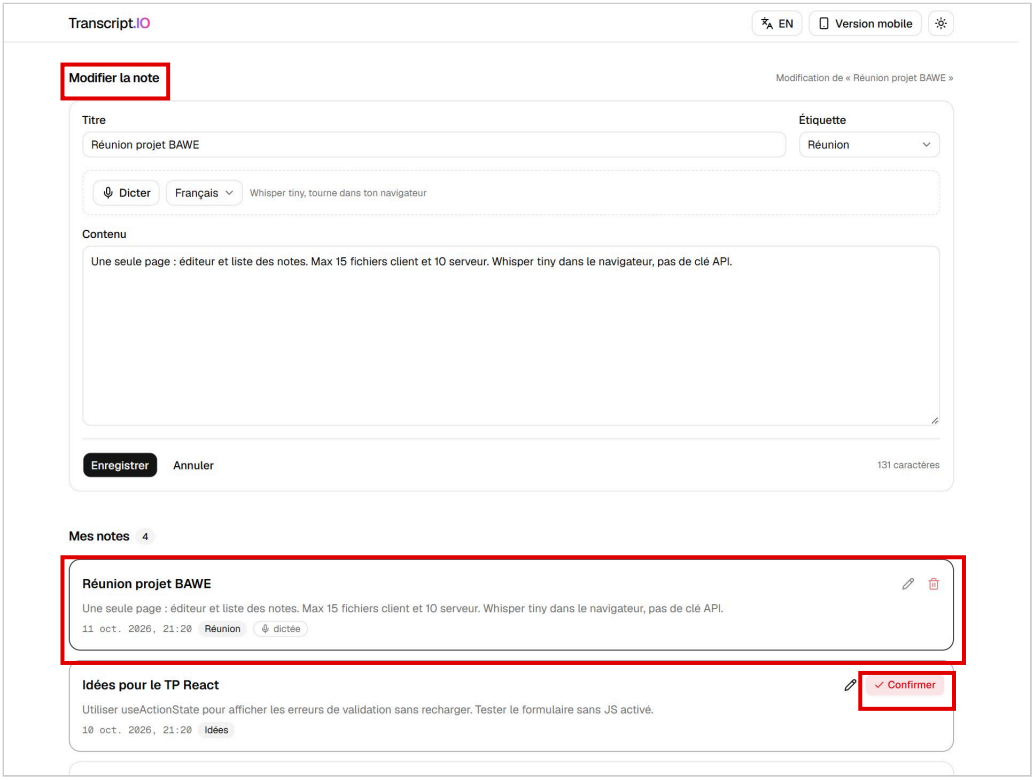


Figure 6 : modification d'une note et confirmation de suppression

Version anglaise

Le bouton **EN** traduit tout le site, y compris les dates et les noms des étiquettes. Il devient **FR** pour revenir en français. Le choix est gardé dans un cookie.

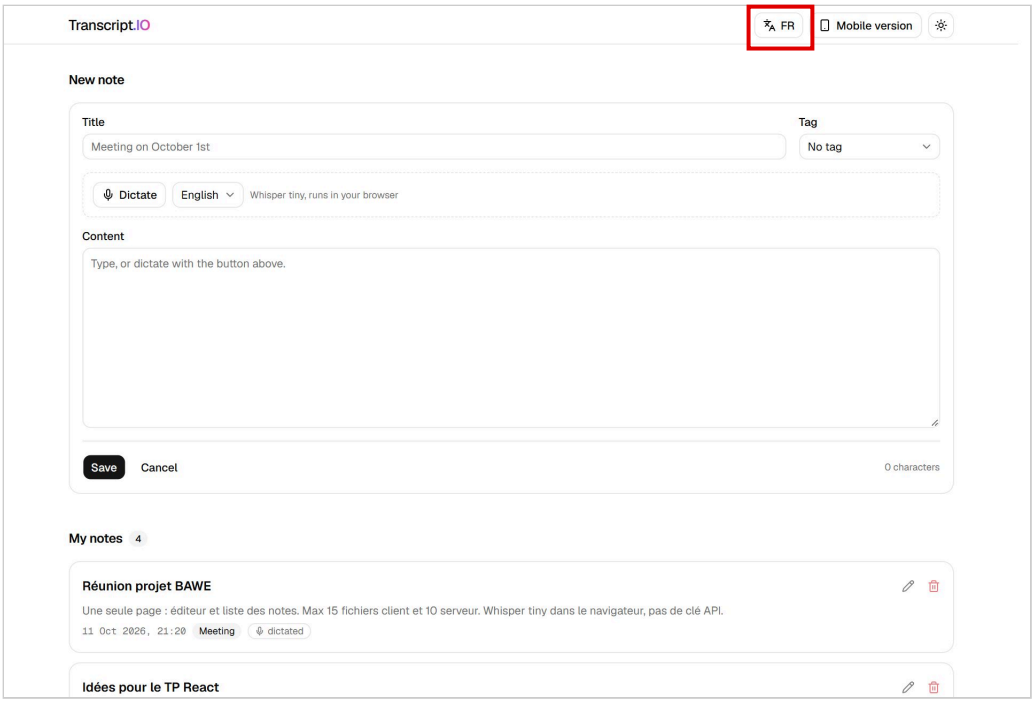


Figure 7 : version bureau en anglais

Version mobile

La version mobile s'affiche sur une seule colonne, avec des boutons plus grands. Les réglages (langue, thème, retour à la version bureau) sont dans un menu caché qu'on ouvre avec le bouton ☰. Sur un téléphone, cette version est choisie automatiquement.

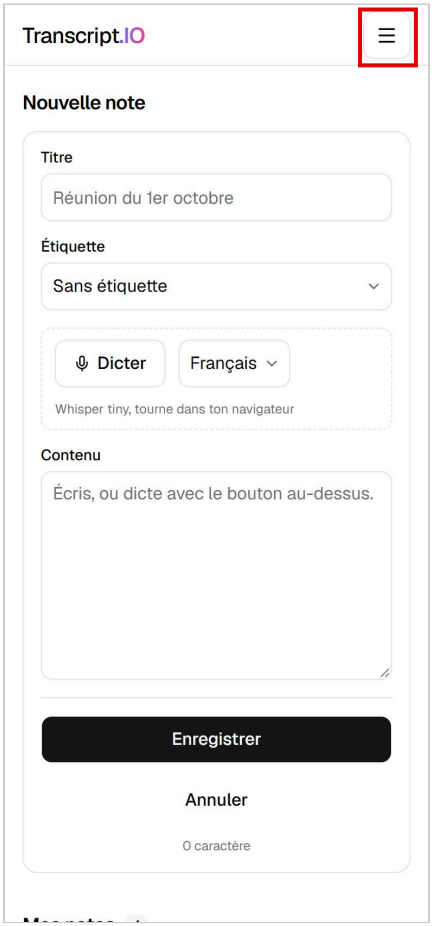


Figure 8 : mobile, français

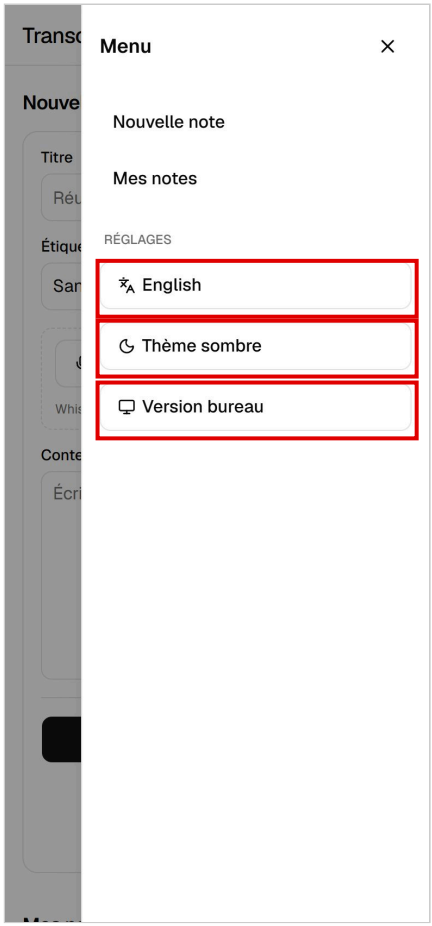


Figure 9 : menu ouvert

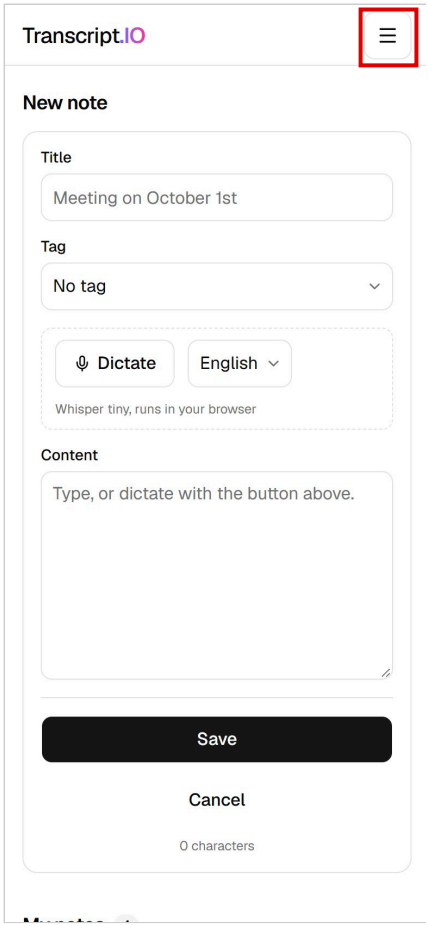


Figure 10 : mobile, anglais

3. Détails techniques

Le bouton mobile / bureau

Normalement, les styles « responsive » dépendent de la largeur de l'écran. Avec ça, on ne peut pas afficher la version mobile sur un ordinateur quand on clique sur un bouton. J'ai donc ajouté un attribut `data-view` sur la balise `<html>`, qui vaut `mobile` ou `desktop`. J'ai créé deux variantes Tailwind, `mob:` et `desk:`, qui ne s'appliquent que si cet attribut a la bonne valeur. Le bouton enregistre le choix dans un cookie, et le serveur renvoie la page avec la bonne version. Si l'utilisateur n'a rien choisi, le serveur regarde le navigateur (user-agent) pour savoir si c'est un téléphone. La langue marche de la même façon, ce qui permet de traduire aussi ce qui est généré côté serveur, comme les dates.

La dictée avec Whisper dans le navigateur

J'ai choisi de faire tourner Whisper dans le navigateur plutôt que d'envoyer l'audio à une API : pas de clé payante, pas de serveur d'IA, et l'audio reste sur l'ordinateur. J'ai utilisé la bibliothèque `browser-whisper`, en me basant sur sa documentation et son exemple Next.js. Le code est dans `hooks/use-whisper.ts`. Les principales difficultés :

- **Le modèle pèse environ 64 Mo.** Il est téléchargé au premier clic sur « Dicter », puis gardé en cache par le navigateur.
- **L'isolation de la page.** Le WebAssembly multi-thread demande les en-têtes `Cross-Origin-Opener-Policy` et `Cross-Origin-Embedder-Policy`, que j'ai ajoutés dans `next.config.ts`.
- **Le format de l'audio.** Whisper attend du son mono à 16 kHz. Je décode l'enregistrement du micro avec un `AudioContext` réglé à 16 kHz, qui fait la conversion tout seul.
- **La barre de téléchargement** reculait parfois. Je garde donc toujours la plus grande valeur reçue.

Sources : documentation de browser-whisper (whisper.tanpreet.xyz/docs) et son exemple Next.js (github.com/tanpreetjolly/browser-whisper-nextjs-demo) ; article sur COOP et COEP sur web.dev (web.dev/articles/coop-coep).